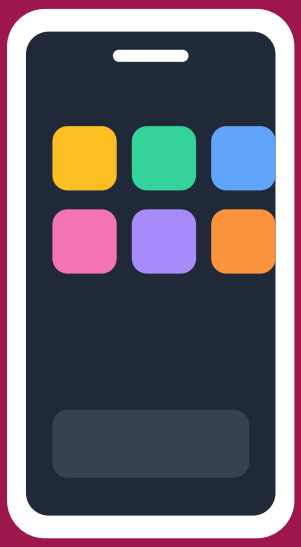

Mobile App Development Guide

Android, iOS, Flutter and React Native: from idea to app store

Choose • Build • Test • Publish • Grow

Updated October 2026

Contents

1. Choosing your approach
2. Setting up your tools
3. Your first app
4. Design, navigation and state
5. Data, networking and backend
6. Device features and permissions
7. Testing and debugging
8. Publishing to the app stores
9. Privacy, monetization and growth
10. Roadmap and launch checklist

> **Who is this for?**
> Beginners and developers from web or desktop who want to build phone apps. You do not need to learn everything here at once: choose one path in Section 1 and follow it.

The mobile app lifecycle

Stage	What happens
Idea and design	Define the problem, sketch screens, build a clickable prototype
Build	Write the app, connect to data and device features
Test	Emulators, real devices, automated tests and beta testers
Publish	Prepare store listings, privacy details and submit for review
Grow and maintain	Monitor crashes, read reviews, ship updates

1. Choosing your approach

There are three ways to build a mobile app. The right choice depends on your skills, budget and the features you need.

Approach	Languages / tools	Strengths	Trade-offs
Native Android	Kotlin, Jetpack Compose, Android Studio	Best performance and newest platform features	Android only
Native iOS	Swift, SwiftUI, Xcode	Best performance and tight Apple integration	iPhone and iPad only; needs a Mac
Cross-platform	Flutter (Dart), React Native / Expo (JavaScript or TypeScript), .NET MAUI (C#), Kotlin Multiplatform	One codebase for Android and iOS; faster and cheaper	Some platform-specific work; slightly behind brand-new features
Web-based	PWA (HTML, CSS, JavaScript), Capacitor / Ionic	Reuses web skills; no store needed for PWAs	Limited device access, especially on iOS

Quick comparison of cross-platform options

Framework	Language	Best for	Notes
Flutter	Dart	Custom, polished UI on every platform	Draws its own widgets; excellent hot reload; also targets web and desktop
React Native + Expo	JavaScript / TypeScript	Web developers; large ecosystem	Uses real native components; Expo simplifies builds and updates
.NET MAUI	C# and XAML	C# and Windows developers	Android, iOS, macOS and Windows from one project
Kotlin Multiplatform	Kotlin	Share business logic, keep native UI	Good for teams with existing native apps

> **Recommendation**
> If you know JavaScript, start with **React Native and Expo**. If you are starting fresh, **Flutter** has a smooth learning curve. Choose **native** when you need top performance or deep platform features.

2. Setting up your tools

Target	Install	Requirements
Android	Android Studio (includes the SDK and emulator)	Windows, macOS or Linux
iOS	Xcode, from the Mac App Store	A Mac is required to build and submit iOS apps
Flutter	Flutter SDK, plus Android Studio or VS Code with the Flutter extension	Check the setup with `flutter doctor`
React Native / Expo	Node.js and a code editor; the Expo Go app on your phone	Android Studio and Xcode only for native builds

Create a project

```
# Flutter
flutter doctor                 # verify your setup
flutter create my_app
cd my_app
flutter run

# React Native with Expo
npx create-expo-app@latest my-app
cd my-app
npx expo start                 # scan the QR code with Expo Go
```

Emulators and real devices

- **Android emulator:** create a virtual device in Android Studio's Device Manager.
- **iOS simulator:** included with Xcode; open it from Xcode or with `open -a Simulator`.
- **Real phone:** enable Developer options and USB debugging on Android; on iPhone, enable Developer Mode and trust your Mac. Real devices reveal performance and touch issues emulators hide.

Helpful extras

- **Git** for version control, **Figma** for designs, and **Postman** or Bruno for testing APIs.
- A free **Firebase** or **Supabase** account if you need sign-in, a database or push notifications.

Windows users
You can build Android and Flutter or React Native apps for Android on Windows. To build and publish iOS apps you need access to a Mac, either your own, a cloud Mac service, or cloud build services such as Expo EAS.

3. Your first app: a tap counter

The same tiny app in four technologies shows how each one thinks. Each keeps a number in **state** and updates the screen when a button is tapped.

Flutter (Dart), in lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: CounterPage()));

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});
  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter')),
      body: Center(child: Text('Tapped $_count times')),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => _count++),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

React Native (Expo)

Replace the contents of the main screen file (`App.js`, or `app/index.tsx` in newer templates).

```
import { useState } from "react";
import { View, Text, Button, StyleSheet } from "react-native";

export default function App() {
  const [count, setCount] = useState(0);
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Tapped {count} times</Text>
      <Button title="Tap me" onPress={() => setCount(count + 1)} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: "center", justifyContent: "center" },
  text: { fontSize: 24, marginBottom: 12 },
});
```

Native Android (Kotlin, Jetpack Compose)

```
@Composable
fun Counter() {
    var count by remember { mutableIntStateOf(0) }
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Tapped $count times")
        Button(onClick = { count++ }) { Text("Tap me") }
    }
}
```

Android Studio adds missing imports when you press Alt+Enter. Call `Counter()` from your activity's `setContent` block.

Native iOS (Swift, SwiftUI)

```
import SwiftUI

struct ContentView: View {
    @State private var count = 0

    var body: some View {
        VStack(spacing: 16) {
            Text("Tapped \(count) times").font(.title)
            Button("Tap me") { count += 1 }
        }
        .padding()
    }
}
```

What all four share

- **Declarative UI:** you describe what the screen should look like for the current state.
- **State:** changing a variable (through `setState`, `useState`, `remember` or `@State`) rebuilds the affected UI.
- **Components:** screens are built from small, reusable pieces (widgets, components, composables, views).
- **Hot reload:** most tools show code changes on the device within a second or two.

Try it
Modify the example: add a reset button, show a message when the count reaches 10, or store the count so it survives app restarts.

4. Design, navigation and state

Design guidelines

Follow each platform's design system so your app feels familiar:

- **Material Design 3** (Android and Flutter): material.io
- **Human Interface Guidelines** (iOS): developer.apple.com/design

Mobile UX principles

- **Thumb-friendly:** keep key actions within easy reach; touch targets at least about 44 by 44 points (iOS) or 48 by 48 dp (Android).
- **One main task per screen** and clear, short labels.
- **Handle all states:** loading, empty, error and offline, not just the happy path.
- **Respect system settings:** dark mode, font scaling, reduced motion.
- **Design for many sizes:** small phones, large phones, tablets, and landscape.

Navigation patterns

Pattern	Use for
Bottom tab bar	3 to 5 top-level sections (home, search, profile)
Stack navigation	Drilling into details and going back
Drawer / side menu	Many secondary destinations
Modal / bottom sheet	Short, focused tasks and confirmations

Navigation libraries: **Flutter** go_router; **React Native** Expo Router or React Navigation; **Android** Navigation Compose; **iOS** NavigationStack.

State management and architecture

As apps grow, separate the UI from logic. A common pattern is **MVVM**: the **View** shows the data, the **ViewModel** holds state and logic, and the **Model/repository** loads and saves data.

Platform	Popular choices
Flutter	Riverpod, Provider, Bloc
React Native	Zustand, Redux Toolkit, TanStack Query
Android	ViewModel, StateFlow, Hilt
iOS	Observation framework, ObservableObject, SwiftData

5. Data, networking and backend

Local storage

Need	Options
Small settings and flags	SharedPreferences (Android), UserDefaults (iOS), AsyncStorage / MMKV (React Native), shared_preferences (Flutter)
Structured data	SQLite via Room (Android), SwiftData or Core Data (iOS), Drift (Flutter), expo-sqlite (React Native)
Secrets and tokens	Keychain (iOS), Keystore (Android), flutter_secure_storage, expo-secure-store
Files and images	App sandbox storage; cache downloaded images

Calling an API

Most apps talk to a server over HTTPS using JSON. Always handle slow or failing connections. Flutter example using the `http` package:

```
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<List<dynamic>> fetchTodos() async {
  final res = await http.get(
    Uri.parse('https://api.example.com/todos'),
  );
  if (res.statusCode != 200) {
    throw Exception('Failed to load: ${res.statusCode}');
  }
  return jsonDecode(res.body) as List<dynamic>;
}
```

Backend options

Option	Good for
Firebase (Auth, Firestore, Cloud Messaging, Crashlytics)	Fast start with sign-in, database, push notifications and analytics
Supabase	Open-source alternative with PostgreSQL and authentication
Your own API (Node.js, Python, .NET)	Full control and custom logic; see the Web Development guide
Appwrite, AWS Amplify, Azure Mobile Apps	Other managed backends

Offline and sync

Phones lose connectivity often. Cache data locally, show saved content immediately, queue changes while offline, and sync when the network returns (**offline-first** design).

6. Device features and permissions

Phones offer sensors and services that web pages rarely reach. Each is exposed through platform APIs or plugins.

Feature	Flutter package	Expo / React Native module
Camera and photos	camera, image_picker	expo-camera, expo-image-picker
Location and maps	geolocator, google_maps_flutter	expo-location, react-native-maps
Push notifications	firebase_messaging	expo-notifications
Biometrics	local_auth	expo-local-authentication
Sharing and files	share_plus, file_picker	expo-sharing, expo-document-picker
Sensors	sensors_plus	expo-sensors
In-app purchases	in_app_purchase	react-native-iap or RevenueCat

Permissions: do it respectfully

- Ask for a permission **at the moment it is needed**, not on first launch.
- Explain **why** before the system prompt appears, and offer a clear alternative if the user refuses.
- Request the **minimum**: approximate location instead of precise when that is enough.
- Declare permissions in the manifest (`AndroidManifest.xml`) and usage descriptions in `Info.plist`; missing descriptions cause crashes or rejection on iOS.
- Handle denial gracefully; never block the whole app.

Background work and notifications

Mobile operating systems limit background activity to save battery. Use platform-approved tools: WorkManager (Android), BGTaskScheduler (iOS), or the framework's background-task plugins. Send **push notifications** only when valuable and let users control them.

> **Respect battery and data**
> Poll less, batch network calls, avoid constant location tracking, and compress images. Users uninstall apps that drain their battery.

7. Testing and debugging

Types of testing

Level	What it checks	Tools
Unit	Logic and calculations	flutter_test, Jest, JUnit, XCTest
Widget / component	A single screen or component renders and reacts correctly	Flutter widget tests, React Native Testing Library, Compose tests
Integration / UI	Full user flows on a device or emulator	integration_test, Detox, Maestro, Espresso, XCUITest
Manual	Real-world feel, gestures, edge cases	Your devices and beta testers

Debugging tools

- **Hot reload and DevTools:** Flutter DevTools, React Native DevTools, Android Studio Profiler, Xcode Instruments.
- **Logs:** `flutter logs`, `adb logcat` (Android), the Xcode console.
- **Network inspection:** proxy tools such as Charles or Proxyman.

What to test before release

- Different screen sizes, OS versions, and at least one low-end device.
- Poor or no network, airplane mode, and switching between Wi-Fi and mobile data.
- Interruptions: incoming calls, notifications, app switching, rotation, low battery.
- Large text, dark mode and screen reader support (TalkBack on Android, VoiceOver on iOS).
- Fresh install, upgrade from the previous version, and denied permissions.

Monitor in production

Add crash and error reporting early: **Firebase Crashlytics** or **Sentry**. Track real performance and usage with analytics, while following privacy rules.

Beta testing

Use **TestFlight** (iOS) and Google Play **internal, closed or open testing** tracks to distribute builds to testers before the public release.

8. Publishing to the app stores

	Google Play	Apple App Store
Developer account	One-time registration fee (about 25 USD)	Apple Developer Program, about 99 USD per year
Build format	Android App Bundle (`.aab`)	Archive uploaded from Xcode (`.ipa`)
Signing	Upload key, and Play App Signing manages the app key	Certificates and provisioning profiles
Beta channel	Internal / closed / open testing	TestFlight
Review	Automated and manual checks; new personal accounts may need a closed test before production	Human review, often 1 to 2 days; follow App Review Guidelines
Tools	Play Console	App Store Connect

Check current fees, account types and testing requirements on each store's site, as they change.

Build commands

```
# Flutter
flutter build appbundle            # Android (.aab)
flutter build ipa                  # iOS (needs a Mac)

# Expo (cloud builds, works from Windows for iOS too)
npm install -g eas-cli
eas build --platform all
eas submit --platform all
```

What you need for the store listing

- App **name**, short and full descriptions, and keywords.
- **Icon** (1024 x 1024 px) and **screenshots** for required device sizes; an optional preview video.
- A **privacy policy URL** and completed privacy forms (Data safety on Google Play, privacy nutrition labels on Apple).
- Age rating, content category and contact details.
- **Version name** (for users, such as 1.2.0) and **build number** that increases with every upload.

Common rejection reasons

- Crashes or broken features during review; incomplete or placeholder content.
- Missing privacy policy or permission explanations; collecting data without disclosure.
- Demo login not provided for apps that require an account.
- Misleading descriptions or screenshots, or copied content.

9. Privacy, monetization and growth

Privacy and compliance

- Collect only the data you need, and explain how it is used.
- Follow laws that apply to your users: **GDPR** (EU), **CCPA/CPRA** (California), and rules for children's apps such as **COPPA**.
- iOS requires user consent for cross-app tracking (**App Tracking Transparency**). Do not track unless you truly need to.
- Offer account deletion inside the app; both stores require it for apps with sign-up.
- Encrypt sensitive data, use HTTPS, and keep tokens in secure storage.

Ways to make money

Model	How it works	Notes
Paid upfront	One-time price	Hard to get downloads without a strong reputation
Subscriptions	Recurring payment for ongoing value	Popular; use RevenueCat or the store billing APIs
In-app purchases	Buy items, features or content	Digital goods generally must use store billing
Freemium	Free basics, paid upgrade	Make the free version genuinely useful
Ads	AdMob and other networks	Needs large user numbers; can hurt experience
Physical goods and services	Use external payment providers	Different rules apply than for digital goods

Growth and keeping users

- **App Store Optimization (ASO):** clear title, keywords, strong icon and screenshots, ratings.
- Make the **first minute** great: a quick onboarding and an early moment of value.
- Use **analytics** to find where users drop off, and fix those screens first.
- Ask for ratings **after** a positive moment, using the system review prompt.
- Reply to reviews, ship regular updates, and keep release notes honest and useful.
- Keep up with OS releases; new Android and iOS versions arrive every year and can break apps.

10. Roadmap and launch checklist

Suggested learning path

Stage	Learn	Build
1. Language basics	Dart, JavaScript/TypeScript, Kotlin or Swift	Console programs and small exercises
2. UI and layout	Widgets/components, layouts, styling, navigation	Multi-screen app: notes or to-do list
3. State and data	State management, local storage, REST APIs	Weather or recipe app using a public API
4. Accounts and cloud	Firebase or Supabase, authentication, push	App with sign-in and synced data
5. Device features	Camera, location, permissions, notifications	Photo journal or fitness tracker
6. Ship it	Testing, signing, store listings, analytics	Publish your app to the stores

Project ideas

- **Starter:** tip calculator, unit converter, habit tracker, flashcards.
- **Intermediate:** expense tracker with charts, recipe finder, workout log, language-learning quiz.
- **Advanced:** chat app, ride or delivery tracker, marketplace with payments, offline-first field-service app.

Pre-launch checklist

- [ ] App tested on several devices, OS versions and screen sizes
- [ ] Offline, error and empty states handled
- [ ] Permissions requested in context with clear explanations
- [ ] Crash reporting and analytics are set up
- [ ] Privacy policy published; store privacy forms completed
- [ ] App icon, screenshots and descriptions ready
- [ ] Version and build numbers set; release build signed
- [ ] Beta test completed through TestFlight or Play testing
- [ ] Support email and a plan for updates and user feedback

Free learning resources

- **docs.flutter.dev** and **docs.expo.dev**: official Flutter and Expo guides.
- **developer.android.com**: Android basics and Jetpack Compose courses.
- **developer.apple.com**: SwiftUI tutorials and Swift documentation.

> **Final advice**
> Ship a small app early. A simple app that is live in the stores teaches you more than months of tutorials, and real users show you what to build next.